

Testing with AI Agents: An Empirical Study of Test Generation Frequency, Quality, and Coverage

Suzuka Yoshimoto[†], Shun Fujita[†], Kosei Horikawa[†], Daniel Feitosa[§], Yutaro Kashiwa[†], Hajimu Iida[†]

[†]Nara Institute of Science and Technology, Japan

[§]University of Groningen, the Netherlands

{yoshimoto.suzuka.yw4,fujita.shun.fp9,horikawa.kosei.hk1,yutaro.kashiwa,iida}@naist.ac.jp
d.feitosa@rug.nl

Abstract

Agent-based coding tools have transformed software development practices. Unlike prompt-based approaches that require developers to manually integrate generated code, these agent-based tools autonomously interact with repositories to create, modify, and execute code, including test generation. While many developers have adopted agent-based coding tools, little is known about how these tools generate tests in real-world development scenarios or how AI-generated tests compare to human-written ones.

This study presents an empirical analysis of test generation by agent-based coding tools using the AIDev dataset. We extracted 2,232 commits containing test-related changes and investigated three aspects: the frequency of test additions, the structural characteristics of the generated tests, and their impact on code coverage. Our findings reveal that (i) AI authored 16.4% of all commits adding tests in real-world repositories, (ii) AI-generated test methods exhibit distinct structural patterns, featuring longer code and a higher density of assertions while maintaining lower cyclomatic complexity through linear logic, and (iii) AI-generated tests contribute to code coverage comparable to human-written tests, frequently achieving positive coverage gains across several projects.

CCS Concepts

• **Software and its engineering** → **Integrated and visual development environments**; **Automatic programming**; *Software evolution*; *Maintaining software*.

Keywords

Agentic Coding, Agents, Software Testing, Coverage

ACM Reference Format:

Suzuka Yoshimoto[†], Shun Fujita[†], Kosei Horikawa[†], Daniel Feitosa[§], Yutaro Kashiwa[†], Hajimu Iida[†]. 2026. Testing with AI Agents: An Empirical Study of Test Generation Frequency, Quality, and Coverage. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793620>

1 Introduction

Software testing is an essential activity that forms the foundation of quality assurance [6, 18, 20]. Code lacking adequate tests cannot

only delay bug detection but also increase the risk of regressions and undermine developers' confidence in making changes [21, 38]. Furthermore, low test coverage can discourage code modifications and extensions, thereby hindering the evolution of projects [19, 34]. To avoid such problems, developers are encouraged to write tests in parallel with feature implementation; however, due to time constraints and the tedious nature of test creation, sufficient tests are often not written in practice [7, 33]. To address this, automated test generation has evolved from traditional techniques like symbolic execution [5] to recent approaches leveraging Large Language Models (LLMs) [15].

More recently, the mainstream approach has been shifting from single-prompt-based methods to approaches utilizing AI agents. In agent-based approaches, by understanding the context of the entire repository and autonomously repeating cycles of test creation, execution, and modification, more sophisticated test generation becomes possible [14]. Although existing studies have evaluated the quality and coverage of LLM-generated tests in controlled experimental environments [32], it is not sufficiently understood to what extent agent-based tools generate tests in actual development settings and how the generated tests differ from those created by humans. This represents an important research challenge for evaluating the practicality of agent-based test generation technology.

This study aims to quantitatively analyze the reality of test generation by agent-based coding tools. Specifically, we extracted commits containing test-related changes from the AIDev dataset [22] and investigated the frequency of test generation, characteristics, and impact on coverage in those commits. We identified 2,232 commits and compared test methods generated by AI with those created by humans, thereby conducting a multifaceted analysis of the effects of test generation by agent-based tools.

2 Data Collection

To identify projects suitable for our analysis, we applied the following steps using the AIDev dataset [22].

1. Language Filtering: We analyzed the AIDev dataset to identify the languages with the most AI-authored pull requests (PRs). We found TypeScript to be one of the most common languages across all agents (22.4% of PRs) and, thus, selected it for this study.

2. Framework Filtering: To measure coverage systematically (for RQ3), we decided to standardize the test framework use to make this study feasible. In particular, we investigated the 2,807 unique repositories from the AIDev dataset that contain PRs, finding that 650 were identified as TypeScript projects, of which 400 had detectable test frameworks. Among these, *vitest* was found to be the



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2474-9/2026/04

<https://doi.org/10.1145/3793302.3793620>

Table 1: Prevalence of AI-Generated Test Addition Commits in Selected Projects.

Project	Test-adding Commits	
	All	AI-authored (%)
cal.com [8]	894	129 (14.4%)
azure-sdk-for-js [4]	366	7 (1.9%)
liam [23]	352	43 (12.2%)
appkit [31]	257	29 (11.3%)
helper [2]	123	13 (10.6%)
primer-react [29]	89	31 (34.8%)
alchemy [1]	76	54 (71.1%)
total-typescript-monorepo [25]	26	26 (100.0%)
llmgateway [35]	25	14 (56.0%)
capgo [9]	24	20 (83.3%)

most prevalent, used in 179 projects (44.8% of projects with test frameworks). Given *vitest*'s dominance in the TypeScript ecosystem within our dataset and its straightforward coverage measurement capabilities, we decided to target *vitest* projects for our analysis.

3. Project Identification: Next, focusing on the identified *vitest* projects, we returned to the AIDev dataset to find those with a high volume of AI-authored test additions. We inspected the patch information of commits within AI-authored PRs. We filtered for PRs that contained new test method definitions (e.g., `describe()`, `it()`) on added lines (prefixed with `+`).

Final Dataset: We selected the top 10 projects with the highest AI-authored test activity to ensure a focus on the most significant contributions. This sample size was determined by the technical complexity of dynamic analysis; measuring test coverage (RQ3) requires manual environment setup and dependency resolution for each repository, making a larger-scale execution impractical.

3 Research Questions

RQ₁: How often do developers add tests with Agent Coding Tools?

Motivation. Understanding the prevalence of AI-generated test contributions is the first step in assessing their overall impact. We examine the proportion of test-added commits among all commits made by AI agents to gauge their activity level in this specific task.

Approach. We analyzed 10 selected TypeScript projects to quantify the contribution of AI agents to test code generation. Our methodology focuses on identifying commits where AI agents explicitly introduced new test methods. The process consists of four steps:

1. *Defining the Temporal Scope.* We determined the analysis period for each project based on the available data in the AIDev dataset. By examining the `pull_request` table, we extracted the minimum and maximum `created_at` timestamps for each repository. This defined interval served as the boundary for our data collection, ensuring that our commit analysis corresponds precisely to the period of AI activity captured in the dataset.

2. *Commit Retrieval and Filtering.* We utilized the GitHub API to retrieve all commits associated with pull requests within the defined period. To isolate substantive code contributions, we filtered this set to exclude merge commits. Furthermore, we narrowed our focus to

commits that modified at least one test file, as these are the potential candidates for test additions.

3. *Detecting Test Method Additions.* Modification of a test file does not necessarily guarantee that a new test was added; it could be a refactor or a deletion. To accurately identify test additions, we performed a granular analysis of the commit patch. We inspected only the added lines (prefixed with `+`) and applied regular expressions to detect the syntax of new test definitions common in the *vitest* and TypeScript ecosystem. Specifically, we looked for patterns such as `test()`, `it()`, `describe()`, `test.skip()`, `test.only()`, `xtest`, and `f.test`. A commit was classified as a "Test Addition Commit" only if its patch contained these specific markers of new test cases.

4. *AI Author Identification.* We determined the authorship of the identified Test Addition Commits by checking the committer name to determine whether the author was an AI or a human. We used AST parsing to extract test methods and `git blame` to trace the authorship of each line within those methods. A commit was classified as AI-authored only if all lines within all of its added test methods were authored exclusively by AI agents.

5. *Metrics calculation.* We computed the proportion of test-adding commits attributed to AI contributors.

Results. Table 1 summarizes the prevalence of AI-authored test additions across the selected projects. The results reveal a highly polarized landscape regarding the role of AI in test generation.

Shift in AI Role by Project Scale. We observe that the role of AI agents in test generation significantly varies with the project's scale, which we characterize by the number of human contributors. In small-scale or early-stage projects with a limited contributor base, AI agents often take a *dominant* role. For instance, in `total-typescript-monorepo` (5 contributors) and `capgo` (28 contributors), AI-generated tests account for 100% and 83.3% of test additions, respectively. In contrast, for large-scale projects with hundreds of contributors, the AI contribution shifts to an *auxiliary* role. In the largest analyzed project, `cal.com` (876 contributors), the AI contribution is consistent but remains at 14.4%. This trend is even more pronounced in established enterprise projects like `azure-sdk-for-js` (608 contributors), where the AI contribution drops to a marginal 1.9%. These results suggest that while AI agents can drive testing in small teams, they mainly serve as supportive tools in large-scale, community-driven, or enterprise development.

High AI Dominance in Specific Projects. In three projects, AI agents are the primary or exclusive drivers of test creation. In `total-typescript-monorepo`, 100% of detected test additions were authored by AI. Similarly, `capgo` (83.3%) and `alchemy` (71.1%) show very high AI participation. This suggests that in certain contexts (possibly smaller, experimental, or agent-centric projects), AI tools can already take over testing responsibility, or developers rely on them enough to delegate the task.

Answer to RQ₁. AI contribution to test generation varies by project type: up to 100% in agent-heavy projects, but typically 10% to 15% in broader open-source development.

RQ₂: What is the difference between AI- and human-created test methods?

Motivation. Understanding whether AI-generated tests differ from human-written ones in structure and content is important, as this impacts their quality and maintainability [27].

Approach. We cloned the repositories of the 10 selected projects and checked out the latest release for each, ensuring our analysis was performed on a consistent, stable snapshot of the codebase. We then identified the authorship of individual test methods following the line-level attribution methodology in Step 4 of the RQ1 Approach (AI Author Identification). Based on this, we classified methods as either AI Test Method (all lines authored by AI) or Human Test Method (any line authored by a human). Finally, we compared the two datasets (AI Test Methods and Human Test Methods) using two techniques.

1. *Metric-Based Analysis:* We calculated metrics for each test method: Effective Lines of Code (eLoC, excluding comments and empty lines), number of assertions (# assertions, e.g., `expect()`), and Cyclomatic Complexity (CC). eLoC and # assertions were calculated using regular expressions. CC was calculated by parsing each test method into an AST using Babel (@babel/parser) and counting decision-point nodes (e.g., `If`, `For`) via @babel/traverse.

2. *Embedding Visualization:* We generated vector embeddings for each test method using a pre-trained CodeBERT model to capture deeper semantic and structural characteristics. We then used t-SNE (t-distributed stochastic neighbor embedding) to visualize these high-dimensional embeddings in 2D space, allowing us to observe whether AI- and human-generated tests form distinct clusters.

Results. Figure 1 presents the distribution of the three metrics for human-written and AI-generated tests. We used the Mann-Whitney U test [24] to assess statistical significance. Of the 10 projects analyzed, 9 contained both AI-authored and human-authored test methods. `total-typescript-monorepo` was excluded because its AI-generated tests had been removed or modified by human developers in the latest release.

Effective eLoC: AI Tests are Longer and More Consistent.

Our analysis reveals that AI-generated tests have a slightly higher median eLoC (12.0) compared to human-written tests (11.0). While the Mann-Whitney U test shows a statistically significant difference ($p < 0.001$), the calculated effect size is negligible (Cliff's $\delta = -0.12$), suggesting the central tendencies are practically similar. However, a key observation lies in the variance and distribution (Figure 1a). Human-written tests exhibit a long tail of outliers with high eLoC (Max: 699), whereas AI tests are confined to a narrower range (Max: 87). This suggests that while typical length is comparable, AI agents generate code with higher consistency, avoiding the “massive” test methods that occasionally occur in human-authored code.

Number of Assertions: More Verification Steps. AI-generated tests are characterized by a higher number of assertions (Figure 1b). The aggregated median is 2.00 for AI compared to 1.00 for humans ($p < 0.001$). This finding suggests that AI agents prioritize verbose setup followed by multiple verification steps within a single method.

While this could be interpreted as thorough verification, it also suggests that AI agents may be prone to the *Assertion Roulette* test smell—a known bad practice where multiple assertions in a single test make it difficult to pinpoint the exact cause of failure [36]. In

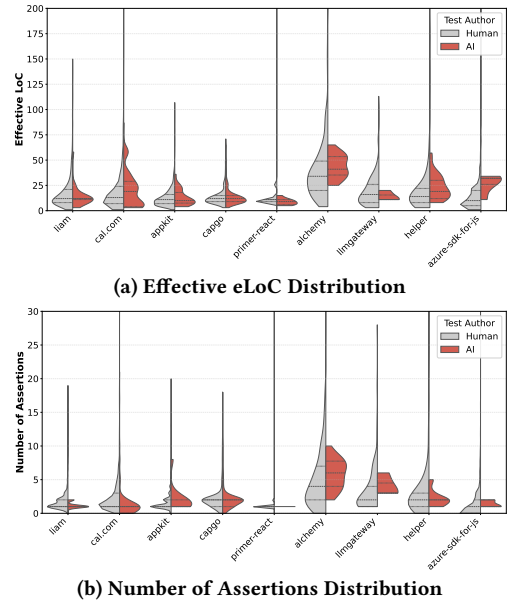


Figure 1: Comparison of Metrics: Human vs. AI Agents

contrast, human developers more frequently adhere to the “one assertion per test” principle, resulting in a lower median count. This structural difference highlights a potential trade-off: AI tests provide high verification density but may increase the maintenance effort required during debugging.

Cyclomatic Complexity: Simpler Logic with Fewer Outliers.

Despite being longer, AI-generated tests do not exhibit higher complexity. Detailed complexity results are available in our online appendix [40]. Significant differences were found in 2 out of 9 projects. The median complexity for both groups is 1.00, but the mean for AI (1.09) is lower than for humans (1.31). Crucially, AI tests rarely exceed a complexity of 2 or 3, lacking the high-complexity outliers seen in human tests. This indicates a strong preference by AI agents for linear, straightforward test logic without complex branching.

Semantic Analysis with Embeddings. To capture characteristics beyond simple metrics, we visualized the semantic representations of the test methods using CodeBERT and t-SNE (Figure 2). By analyzing the distribution of AI-authored tests in the semantic space, we identified three distinct patterns of generation based on the spatial relationship between AI-generated and human-authored tests: (i) **Dispersed Distribution**, where AI tests are broadly scattered throughout the entire human test distribution, indicating structural similarity to various human styles (e.g., `appkit`, `cal.com`, and `capgo`); (ii) **Localized Distribution**, where AI tests overlap with only specific clusters or regions of human tests (e.g., `alchemy`, `primer-react`, `helper`); and (iii) **Isolated Distribution**, where AI tests are concentrated in limited regions that are spatially separated from most human tests (e.g., `llmgateway`). Visualization results for all projects are available in our replication package [41].

Answer to RQ₂. AI-generated tests tend to be longer with more assertions, yet exhibit lower or equal complexity, suggesting a preference for linear logic over complex branching.

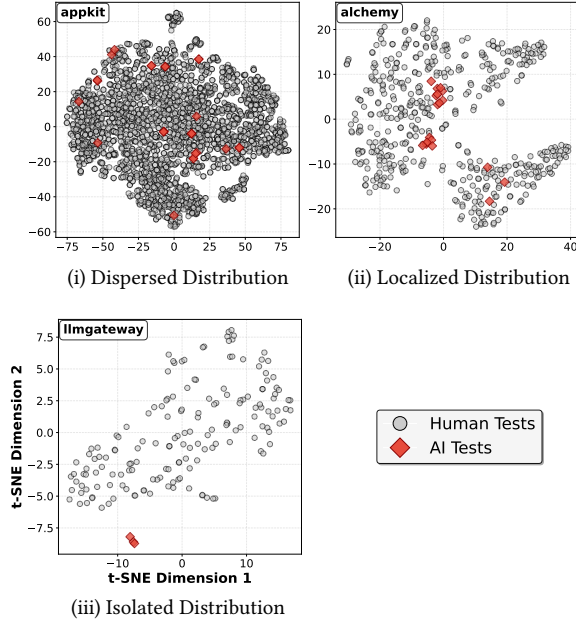


Figure 2: t-SNE Visualization of CodeBERT Embeddings.

RQ₃: Do Agent Coding Tools improve test coverage?

Motivation. While adding tests is one thing, their practical value is often measured by their ability to exercise untested code [16]. We investigate the tangible impact of AI-generated tests by measuring the extent to which they contribute to the project’s code coverage.

Approach. To assess the impact of AI-generated tests on project quality, we measured coverage changes before and after test-adding commits. For each test-adding commit identified in RQ1, we calculated coverage at two points: the parent commit and the test-adding commit itself. Comparing these metrics allowed us to assess how AI-authored tests contributed to coverage improvement.

Results. Table 2 summarizes the impact of AI-authored and human-authored test additions on code coverage. Of 10 projects analyzed in RQ1 and RQ2, we obtained coverage data for 531 test-adding commits across 3 projects where automated execution was feasible (other projects require project-specific setup).

In *liam* and *appkit*, AI-authored tests contributed positively to statement coverage, with average increases of +0.072 pts and +0.030 pts, respectively. In *liam*, among the 12 AI commits that changed statement coverage, 9 commits (75%) resulted in improvement. In *appkit*, AI-authored tests showed more substantial improvement in branch coverage (+0.183 pts) compared to human-authored tests (+0.008 pts). For *helper*, AI-authored tests showed nearly no changes in statement and branch coverage, while human contributions led to a +0.213 pts increase in branch coverage.

Answer to RQ₃. AI agents improve code coverage at levels comparable to or exceeding human developers. In *liam* and *appkit*, AI-generated tests yielded higher statement and branch coverage gains than humans.

Table 2: Average Coverage Change

Project	Author	Statement	Branch
liam	AI ($N = 18$)	+0.072	+0.074
	Human ($N = 143$)	-0.090	-0.142
appkit	AI ($N = 28$)	+0.030	+0.183
	Human ($N = 221$)	-0.001	+0.008
helper	AI ($N = 13$)	0.000	+0.006
	Human ($N = 108$)	0.000	+0.213

4 Related Work

Automated test generation has evolved significantly, shifting from traditional Search-Based Software Testing to LLM-driven approaches [26, 32]. Traditional tools like EvoSuite [13] excel at maximizing coverage but often produce unreadable code [3]. While recent LLM-based methods have improved readability [12], simple prompt-based approaches suffer from hallucinations and lack project-wide context [43], frequently leading to non-compilable code [42].

To address these limitations, recent research has focused on agent-based generation, demonstrating its superior performance through iterative refinement [10, 28, 37]. Studies indicate that agents employing self-correction loops significantly outperform single-turn LLMs [11, 30]. Furthermore, comparative evaluations reveal that Code Agents designed for repair can effectively generate reproducing tests and achieve high coverage [39].

However, these evaluations are largely limited to controlled benchmarks such as SWT-Bench [17], leaving the prevalence and quality of agent-generated tests in the wild unexplored. To address this gap, we analyze the AIDev dataset [22], examining agent-generated tests in actual development.

5 Conclusion

This study presented an empirical analysis of test generation by AI agents using the AIDev dataset. Our investigation of 2,232 test-related commits revealed that AI agents authored 16.4% of all test-adding commits, demonstrating their significant role in modern software development. Structural analysis showed that AI-generated tests feature longer code and higher assertion density than human-written tests, while maintaining lower cyclomatic complexity through linear logic. AI-generated tests achieve code coverage gains comparable to human-authored ones, though their testing scope tends to be more localized in complex contributions.

Future research should fully evaluate long-term quality. First, *mutation testing* should determine if the high assertion density of AI tests translates into superior fault-detection capabilities. Second, systematic investigation of *test smells* (e.g., Assertion Roulette) is essential to assess whether AI-generated patterns introduce technical debt that could hinder future maintenance.

Replication Packages: We provide the program and data used in our replication package [41].

Acknowledgments

This work was supported by JSPS KAKENHI (JP24K02921, JP25K21359) and JST PRESTO (JPMJPR22P3), ASPIRE (JPMJAP2415), and AIP Accelerated Program (JPMJCR25U7).

References

- [1] alchemy-run. 2025. Alchemy. GitHub repository. <https://github.com/alchemy-run/alchemy> v0.76.1.
- [2] antiwork. 2025. Helper. GitHub repository. <https://github.com/antiwork/helper>
- [3] Andrea Arcuri. 2018. An experience report on applying software testing academic results in industry: we need usable automated test generation. *Empir. Softw. Eng.* 23, 4 (2018), 1959–1981.
- [4] Azure. 2025. Azure SDK for JavaScript. GitHub repository. <https://github.com/Azure/azure-sdk-for-js>
- [5] Roberto Baldoni, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. *ACM Comput. Surv.* 51, 3 (2018), 50:1–50:39.
- [6] Kent L. Beck. 2003. *Test-driven Development - by example*. Addison-Wesley.
- [7] Moritz Beller, Georgios Gousios, and Andy Zaidman. 2015. How (Much) Do Developers Test?. In *Proc. of the 37th IEEE International Conference on Software Engineering (ICSE)*. 559–562.
- [8] calcom. 2025. Cal.com. GitHub repository. <https://github.com/calcom/cal.com>
- [9] Cap-go. 2025. Capgo. GitHub repository. <https://github.com/Cap-go/capgo>
- [10] Guangyao Chen, Siwei Dong, Yu Shu, Ge Zhang, Jaward Sesay, Börje Karlsson, Jie Fu, and Yemin Shi. 2024. AutoAgents: A Framework for Automatic Agent Generation. In *Proc. of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. 22–30.
- [11] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *Proc. of The 12th International Conference on Learning Representations (ICLR)*.
- [12] Amirhossein Deljouy, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. 2025. Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests. In *Proc. of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1449–1461.
- [13] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proc. of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering (ESEC/FSE)*. 416–419.
- [14] Anusha Garlapati, M N V Satya Sai Muni Parmesh, Savitha, and Jaisri S. 2024. AI-Powered Multi-Agent Framework for Automated Unit Test Case Generation: Enhancing Software Quality through LLM's. In *Proc. of the 5th IEEE Global Conference for Advancement in Technology (GCAT)*. 1–5.
- [15] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8 (2024), 220:1–220:79.
- [16] Marko Ivankovic, Goran Petrovic, René Just, and Gordon Fraser. 2019. Code coverage at Google. In *Proc. of 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 955–963.
- [17] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *Proc. of the 12th International Conference on Learning Representations (ICLR)*.
- [18] Ali Khatami and Andy Zaidman. 2023. Quality Assurance Awareness in Open Source Software Projects on GitHub. In *Proc. of the 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 174–185.
- [19] Miryung Kim, Thomas Zimmermann, and Nachiappan Nagappan. 2014. An Empirical Study of Refactoring Challenges and Benefits at Microsoft. *IEEE Transactions on Software Engineering (TSE)* 40, 7 (2014), 633–649.
- [20] Amy J. Ko, Bryan Dosono, and Neeraja Duriseti. 2014. Thirty years of software problems in the news. In *Proc. of the 7th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. 32–39.
- [21] Fabian Leinen, Daniel Elsner, Alexander Pretschner, Andreas Stahlbauer, Michael Sailer, and Elmar Jürgens. 2024. Cost of Flaky Tests in Continuous Integration: An Industrial Case Study. In *Proc. of 2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 329–340.
- [22] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Team-mates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *CoRR* abs/2507.15003 (2025).
- [23] liam-hq. 2025. Liam ERD. GitHub repository. <https://github.com/liam-hq/liam>
- [24] H. B. Mann and D. R. Whitney. 1947. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *Annals of Mathematical Statistics* 18 (1947), 50–60.
- [25] mattpocock. 2025. Total TypeScript Monorepo. GitHub repository. <https://github.com/mattpocock/total-typescript-monorepo>
- [26] Phil McMinn. 2011. Search-Based Software Testing: Past, Present and Future. In *Proc. of 2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops (ICST)*. 153–163.
- [27] Gerrard Meszaros. 2007. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley Professional, Boston.
- [28] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin T. Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Proc. of the 38th Annual Conference on Neural Information Processing Systems (NIPS)*.
- [29] primer. 2025. Primer React. GitHub repository. <https://github.com/primer/react>
- [30] Matthew Renze and Erhan Guven. 2024. Self-Reflection in Large Language Model Agents: Effects on Problem-Solving Performance. In *Proc. of the 2nd International Conference on Foundation and Large Language Models (FLLM)*. 516–525.
- [31] reown-com. 2025. AppKit. GitHub repository. <https://github.com/reown-com/appkit>
- [32] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Trans. Software Eng.* 50, 1 (2024), 85–105.
- [33] Lucinéia Souza, Sávio Freire, Verusca Rocha, Nicolli Rios, Rodrigo O. Spínola, and Manoel G. Mendonça. 2020. Using Surveys to Build-up Empirical Evidence on Test-Related Technical Debt. In *Proc. of the 34th Brazilian Symposium on Software Engineering (SBES)*. 750–759.
- [34] Davide Spadini, Fabio Palomba, Andy Zaidman, Magiel Bruntink, and Alberto Bacchelli. 2018. On the Relation of Test Smells to Software Code Quality. In *Proc. of 2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 1–12.
- [35] theopenco. 2025. LLM Gateway. GitHub repository. <https://github.com/theopenco/llmgateway>
- [36] A. Van Deursen, L. Moonen, A. van den Bergh, and G. Kok. 2001. Refactoring test code. In *Proc. of the 2nd International Conference on Extreme Programming and Flexible Processes in Software Engineering (XP2001)*. 92–95.
- [37] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2025. Mixture-of-Agents Enhances Large Language Model Capabilities. In *Proc. of The 13th International Conference on Learning Representations (ICLR)*.
- [38] Titus Winters, Tom Manshreck, and Hyrum Wright. 2020. *Software Engineering at Google - Lessons Learned from Programming Over Time*. O'Reilly. Chapter 11: Testing Overview..
- [39] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proc. of the 38th International Conference on Neural Information Processing Systems (NIPS)*.
- [40] Suzuka Yoshimoto. 2025. Cyclomatic Complexity Visualization. https://github.com/suu-y/AI-human-test-comparison/blob/master/figures/cross_project_complexity_by_project.png. Accessed: January 26, 2026.
- [41] Suzuka Yoshimoto and Shun Fujita. 2025. Replication Package. <https://github.com/suu-y/AI-human-test-comparison>. Accessed: January 26, 2026.
- [42] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *CoRR* abs/2305.04207 (2023).
- [43] Jiyang Zhang, Yu Liu, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2025. exLong: Generating Exceptional Behavior Tests with Large Language Models. In *Proc. of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1462–1474.